

Архитектура Channelbox : компоненты и потоки данных

Channelbox

Exported on 09/19/2026

Table of Contents

1. Обзор	7
1.1 Схема: слой клиентов и контур виджета.....	7
1.2 Схема: общая топология.....	8
2. Компоненты.....	9
2.1 Channelbox (Supabase Self-Hosted)	9
2.2 Widget External Services (WES)	11
2.3 Token Service.....	12
2.4 Mail External Service (MES)	13
2.5 MCP-сервер (инструменты для AI-агентов).....	14
2.6 Клиентские приложения	15
3. Потоки данных.....	16
3.1 Без Token Service (прямой режим)	16
3.2 С Token Service (токенизация PII).....	16
3.3 Email-обращение через MES	17
3.4 Массовая рассылка в личный кабинет (WES)	18
3.5 Вызов инструмента внешним AI-агентом (MCP)	18
4. Конфигурация виджета	19
5. Сетевые требования	21
5.1 Домены и разделение endpoints	21
5.2 Порты.....	21
5.3 SSL-сертификаты	22
5.4 Взаимодействие между компонентами	22
5.5 Зоны доступа и VPN	23
5.6 Интеграция с внешними каналами и сервисами (Firewall)	24
Входящие webhook-и (внешний сервис → Channelbox)	24
Исходящие запросы (Channelbox → внешние сервисы).....	25
Правила Firewall	26

7. Интеграции.....	27
7.1 Embed / Iframe (Mdbit Workspace).....	28
7.2 Outbound Webhooks и System Webhooks.....	29
Outbound Webhooks (бизнес-события).....	29
System Webhooks (инфраструктурные события)	29
7.3 Внешний роутер (External Router).....	30
7.4 Operator Status API.....	30
7.5 Email-канал.....	31
Входящие письма.....	31
Исходящие письма.....	32
7.6 Боты и AI-провайдеры.....	32
Внешние боты (Callback API).....	32
AI-провайдеры (LLM)	33
7.7 Stored Data API – выгрузка данных для BI и CRM.....	33
Назначение	33
Авторизация.....	33
Эндпоинты.....	33
Справочники (полная выгрузка).....	33
Диалоги (инкрементальная выгрузка)	34
Посетители (инкрементальная выгрузка)	34
Статистика (агрегаты за день)	34
Служебные.....	35
Паттерн инкрементальной синхронизации (CDC).....	35
Анонимизация PII	35
Безопасность	35
7.8 Broadcast API – массовые рассылки.....	36
7.9 MCP-протокол	36
7.10 Feedback Form API – внешние формы обратной связи	36
7.11 Operator Admin API – управление учётными записями.....	37

7.12 CRM Screen-Pop — автооткрытие карточки клиента	37
7.13 Consent API — согласия 152-ФЗ / GDPR.....	38
7.14 BI-слой: Apache Superset.....	38
8. API-методы платформы (сводная матрица)	39
9. Эксплуатация self-hosted: ограничения и обязательные настройки.....	41
10. Наблюдаемость, SIEM и резервное копирование	42
10.1 Наблюдаемость.....	42
10.2 SIEM	42
10.3 Резервное копирование.....	43
11. Sizing	44
04. Database Schema.....	45
Категории таблиц	45
Core Tables (8).....	46
Configuration Tables (14)	46
Routing Tables (3).....	47
Logging Tables (7).....	47
Operators Tables (13).....	47
SLA Tables (3).....	48
Bots / MCP Tables (9)	48
Knowledge Base Tables (6)	48
GDPR/Consent Tables (3)	49
Security Tables (10).....	49
AI Providers Tables (2)	49
Feedback Tables (7)	49
Embed Tables (2).....	50
Custom Fields Tables (3)	50
Email Tables (1)	50
Mail Worker Tables (6)	50
Borrower Profile Tables (3)	50

Widget / WES Tables (4)	51
Outbound Webhooks Tables (2)	51
Dashboard/Workspace Tables (3).....	51
Templates Tables (3).....	51
Other Tables (6)	51
Broadcasts Tables (6)	52
Views (11).....	53
ER-диаграмма (Core)	53

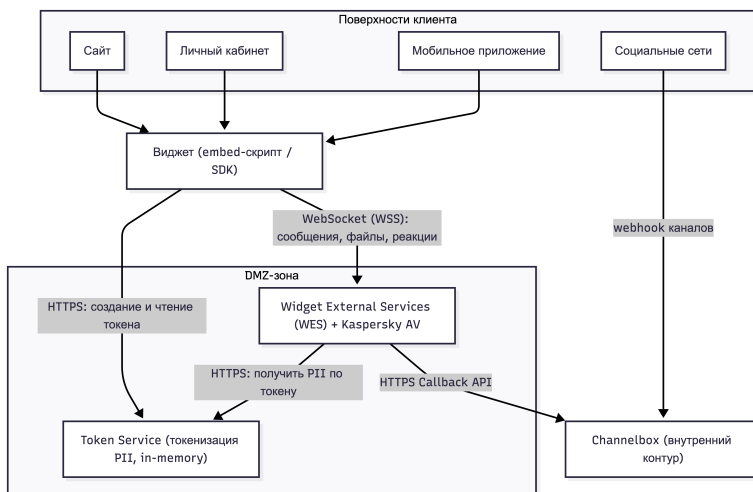
Версия: 2.1 · **Дата:** Август 2026 · Документ пригоден для передачи заказчику.

1. Обзор

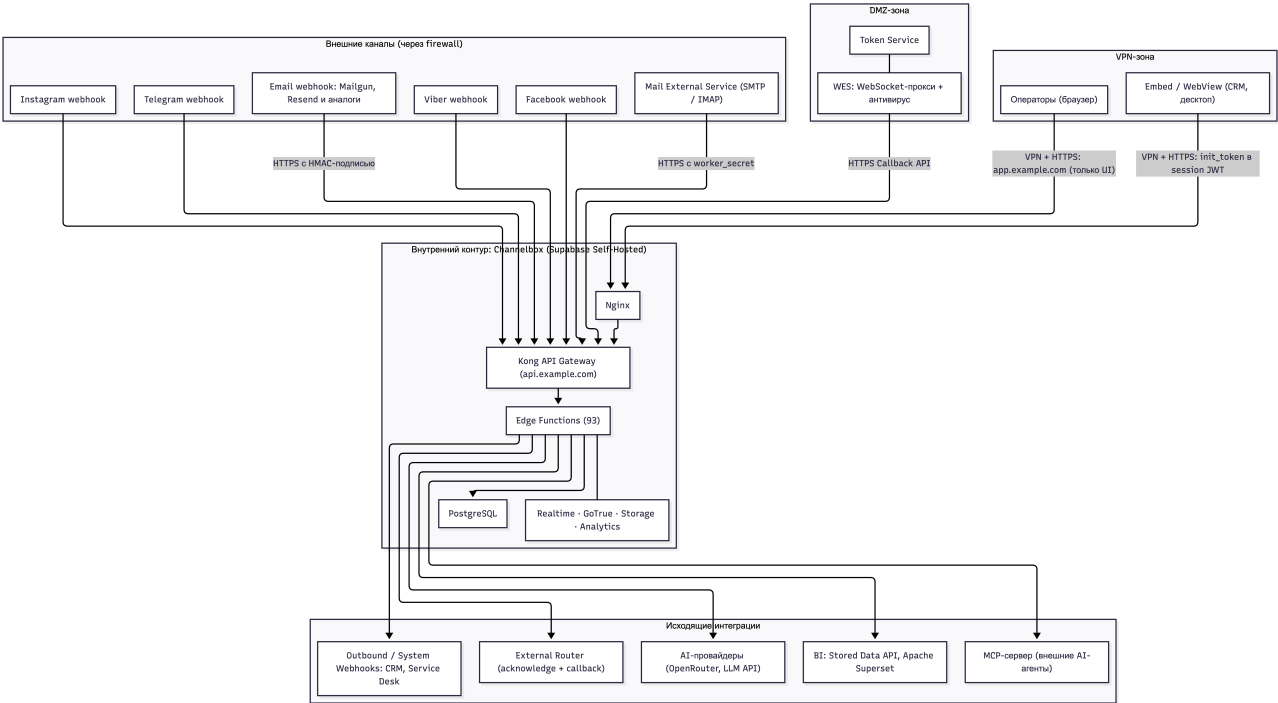
Платформа состоит из шести независимых компонентов, взаимодействующих по HTTPS и WebSocket: Channelbox (ядро), Widget External Services (WES), Mail External Service (MES), Token Service, MCP-сервер (инструменты для внешних AI-агентов) и клиентские приложения. Доступ операторов осуществляется **только через VPN**, а внешние каналы (мессенджеры, соцсети, Email) подключаются через firewall с фиксированными endpoints. Платформа предоставляет интеграционный слой для встраивания UI (Embed/WebView), маршрутизации через внешний роутер, отправки событий в CRM, массовых рассылок, выгрузки данных в BI (Apache Superset) и взаимодействия с AI-провайдерами.

1.1 Схема: слой клиентов и контур виджета

Обращения приходят с четырёх поверхностей клиента: сайт, личный кабинет, мобильное приложение и социальные сети. Виджет держит постоянное **WebSocket**-соединение с WES и отдельно, по **HTTPS**, получает токен в Token Service. WES и Token Service размещены в **DMZ**; во внутренний контур идёт только HTTPS Callback API.



1.2 Схема: общая топология



2. Компоненты

2.1 Channelbox (Supabase Self-Hosted)

Ядро платформы — 13 Docker-контейнеров, развёрнутых через `docker-compose.yml` :

Контейнер	Назначение
Nginx	Reverse proxy, SSL-терминация
Kong	API Gateway, маршрутизация запросов
Edge Functions	93 серверные функции через единый роутер (<code>/functions/v1/{name}</code>)
PostgreSQL	Основная БД (127 таблиц, 11 представлений)
Realtime	WebSocket-подписки на изменения в БД
GoTrue	Аутентификация и управление сессиями
Storage	Файловое хранилище (вложения, аватары)
Analytics	Сбор метрик и логов
Meta	Метаданные PostgreSQL
Rest (PostgREST)	REST API к БД
Dashboard	Панель администрирования Supabase
Imgproxy	Трансформация изображений
Vector	Агрегация логов

Ключевые функции Channelbox:

- Управление проектами и каналами виджетов
- Конфигурация внешнего виджета (цвета, тексты, embed-скрипт)
- Маршрутизация сообщений между операторами и посетителями
- Хранение истории переписки и аналитики

Группировка Edge Functions (93 шт.):

Группа	Примеры функций	Назначение
Каналы	telegram-webhook , viber-webhook , instagram-webhook , email-webhook , send-external	Приём и отправка сообщений во внешних каналах
Виджет / WES	widget-external-service-api , widget-api	Регистрация каналов, конфигурация, embed-скрипт
Почта (MES)	mail-external-service-api , smtp-gateway-api	Обмен с внешним почтовым воркером
Маршрутизация	chat-router , routing-assign , routing-operator-status	Внутренний и внешний роутер, статусы операторов
Жизненный цикл чата	chat-events , message-events , таймауты и пост-обработка	События, таймауты, wrap-up
Боты и AI	bot-gateway , bot-process-message , bot-test , mcp-server	Внешние боты, LLM, MCP-инструменты
Рассылки	broadcast-worker , send-external	Массовые рассылки и импорт списков

Интеграции и API	outbound-webhook-dispatch, stored-data-api, operators-admin-api, embed-token, Consent API	Внешние API-поверхности
SLA и отчёты	sla-check-breaches, экспорт отчётов	SLA-мониторинг, выгрузки
Обслуживание	cron- и cleanup-функции, warmup	Регламентные задачи pg_cron

2.2 Widget External Services (WES)

Внешний микросервис-прокси, обеспечивающий двустороннюю связь между клиентскими виджетами и Channelbox.

Протоколы:

- **К клиентам** — WebSocket (WSS): постоянное двустороннее соединение с виджетом на сайте, в личном кабинете и в мобильном приложении
- **К Channelbox** — HTTPS Callback API: POST {host}/widget/callback/{channel_id}
- **К Token Service** — HTTPS: получение расшифрованных PII по токenu, который прислал виджет

Размещение и безопасность:

- **DMZ-зона** — WES изолирован от внутренней сети, принимает внешние WebSocket-подключения из Интернета и проксирует трафик к Channelbox во внутреннем контуре
- **Антивирусная защита** — Kaspersky Antivirus проверяет все передаваемые файлы (вложения, изображения) перед пересылкой в Channelbox; файлы, не прошедшие проверку, блокируются

Возможности:

- Приём и отправка текстовых сообщений, вложений, реакций
- Поддержка lead-форм и обратной связи
- Системные уведомления (оператор печатает, статусы)
- Управление метаданными посетителя
- Два режима работы: **с Token Service** и **без Token Service**
- **Режим массовых рассылок** — приём канонического payload external_app для доставки уведомлений в личный кабинет клиента

Studio-конфигуратор WES (в панели администратора):

Возможность	Описание
-------------	----------

Расширенный конфиг	140+ полей: внешний вид, тексты, автоприглашения, lead-формы, кастомные поля, видимость по устройствам
Медиа-ассеты	Логотипы, постеры, видео и fallback-изображения — ссылкой или загрузкой в хранилище (бакет <code>widget-assets</code>)
Темы	Набор готовых предустановленных тем, включая тёмные
Превью	Живое превью виджета с реальным <code>project_id</code> и WES-хостом
Область действия	Часть полей задаётся на уровне проекта, часть — на уровне канала

Для self-hosted публичный адрес хранилища задаётся параметром `public_storage_base_url`, иначе браузер получает Mixed Content либо недоступный внутренний хост.

Callback-формат (WES → Channelbox):

При передаче данных от WES в Channelbox используется формат callback-запроса, содержащий: идентификатор получателя (посетителя), тип сообщения (текстовое, системное, обратная связь или запрос оператора), секретный ключ канала для аутентификации, данные отправителя (имя, идентификатор, email оператора), идентификатор канала, текст сообщения, а также тип действия — новое сообщение, ответ, редактирование, удаление или реакция.

2.3 Token Service

In-Memory сервис токенизации персональных данных (PII). Обеспечивает маскировку PII при передаче через внешние каналы связи.

Технический стек: Python 3.11+ · FastAPI · orjson · In-Memory (RWLock)

Характеристики:

- RPS: 147+ (готов к 1 000+)
- Latency: < 100 мс на операцию
- ~1 КБ памяти на токен
- TTL: от 30 минут до 24 часов
- Автоматическая очистка устаревших токенов

API (v1):

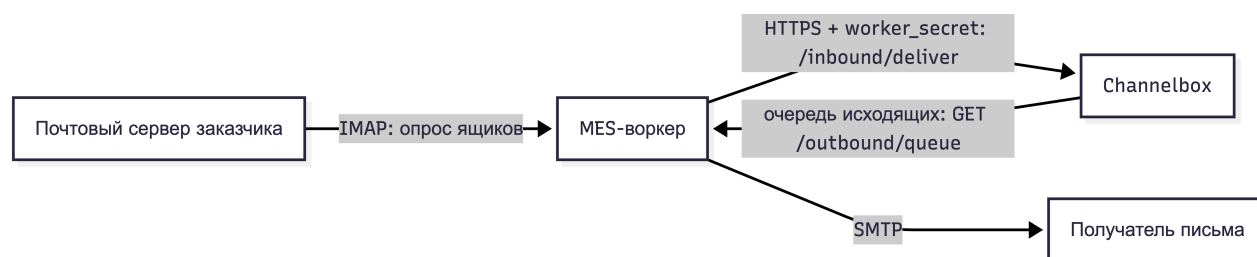
Метод	Эндпоинт	Назначение
POST	/api/v1/tokens	Создание токена (UUID v4)

GET	/api/v1/tokens/{token}	Получение данных по токenu
PUT	/api/v1/tokens/{token}/extend	Продление TTL
DELETE	/api/v1/tokens/{token}	Инвадация токена
GET	/api/v1/health	Проверка состояния
GET	/api/v1/stats	Статистика хранилища

Аутентификация: API-ключ в заголовке запроса.

2.4 Mail External Service (MES)

Внешний почтовый воркер для приёма и отправки email в инфраструктуре заказчика (SMTP/IMAP), без обращения к облачным провайдерам.



Эндпоинты API (/functions/v1/mail-external-service-api):

Эндпоинт	Назначение
POST /health/heartbeat	Регистрация состояния воркера
GET /config/domains	Обслуживаемые домены и настройки
GET /inbound/mailboxes	Перечень ящиков для опроса
POST /inbound/check	Проверка обращения перед доставкой

POST /inbound/deliver	Доставка входящего письма с вложениями
GET /outbound/queue	Забор очереди исходящих писем

Безопасность: у каждого хоста собственный `worker_secret` с ротацией из UI; флаг `allow_private_host` разрешает внутренние адреса только в self-hosted-контуре — в облаке SSRF-проверка остаётся строгой.

Обработка тем писем: каждое письмо сохраняет собственную тему, тема отображается в карточке обращения и передаётся внешним ботам в `email_context`.

2.5 MCP-сервер (инструменты для AI-агентов)

Edge Function `mcp-server` реализует протокол MCP (JSON-RPC) и предоставляет внешним AI-ассистентам (ChatGPT, Claude, Cursor, собственные агенты) управляемый набор инструментов над платформой.

Инструменты (23):

Категория	Инструменты
Жизненный цикл чата	<code>chat_take</code> , <code>chat_close</code> , <code>chat_hold</code> , <code>chat_resume</code> , <code>chat_reopen</code> , <code>chat_return_to_queue</code> , <code>chat_transfer_to_operator</code>
Сообщения	<code>chat_message_send</code> , <code>chat_message_send_from_template</code> , <code>chat_history_get</code>
Теги	<code>chat_tags_get</code> , <code>chat_tag_assign</code> , <code>chat_tag_unassign</code>
Кастомные поля	<code>custom_fields_get</code> , <code>custom_field_set</code> , <code>custom_field_clear</code>

Шаблоны	<code>message_templates_list</code> , <code>message_template_get</code>
Очереди	<code>operator_queues_list</code> , <code>queue_chats_list</code>
Профиль заёмщика	<code>borrower_profile_get</code> , <code>borrower_profile_summary</code> , <code>borrower_service_action</code>

Характеристики: авторизация по токену с ротацией, валидация аргументов по JSON Schema, детальные коды ошибок в ответе инструмента, логирование каждого вызова.

2.6 Клиентские приложения

Конечные точки размещения виджета — сайт, личный кабинет, мобильное приложение. Обращения из социальных сетей приходят не через виджет, а через webhook соответствующего канала (Instagram, Facebook).

Подключение виджета:

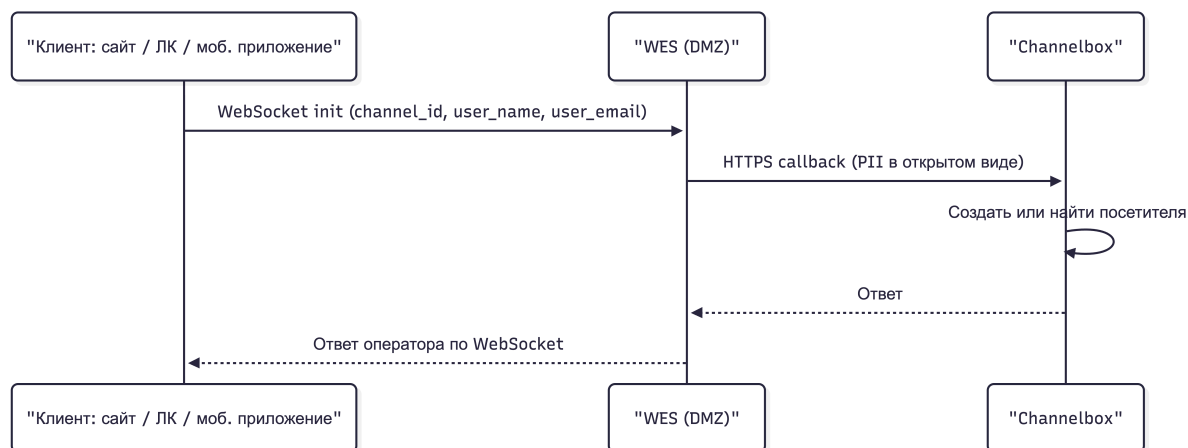
1. Embed-скрипт генерируется в Channelbox (параметры: `project_id`, `lang`, `mode`, `auth_mode`)
2. Виджет подключается к WES по WebSocket (WSS), а к Token Service — по HTTPS
3. Обмен сообщениями в формате JSON (типы: `message`, `edit`, `delete`, `reply`, `reaction`, `feedback`)

Конфигурация из Channelbox:

- Цвета и стили виджета
- Тексты приветствий и заглушек
- Настройки согласий (GDPR/152-ФЗ)
- Привязка к каналу и проекту

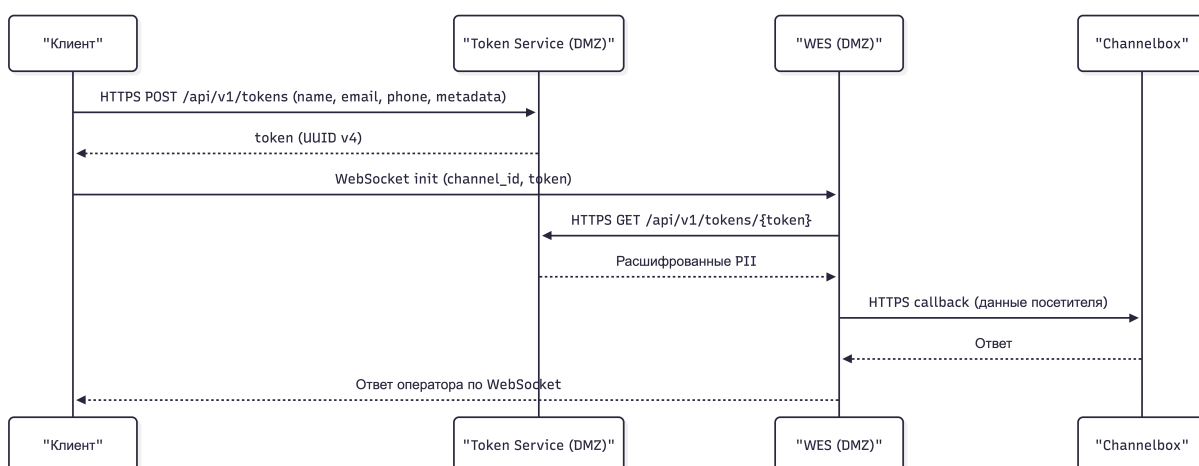
3. Потоки данных

3.1 Без Token Service (прямой режим)



⚠ PII передаётся в открытом виде. Подходит для внутренних сетей или при наличии шифрования на транспортном уровне.

3.2 С Token Service (токенизация PII)

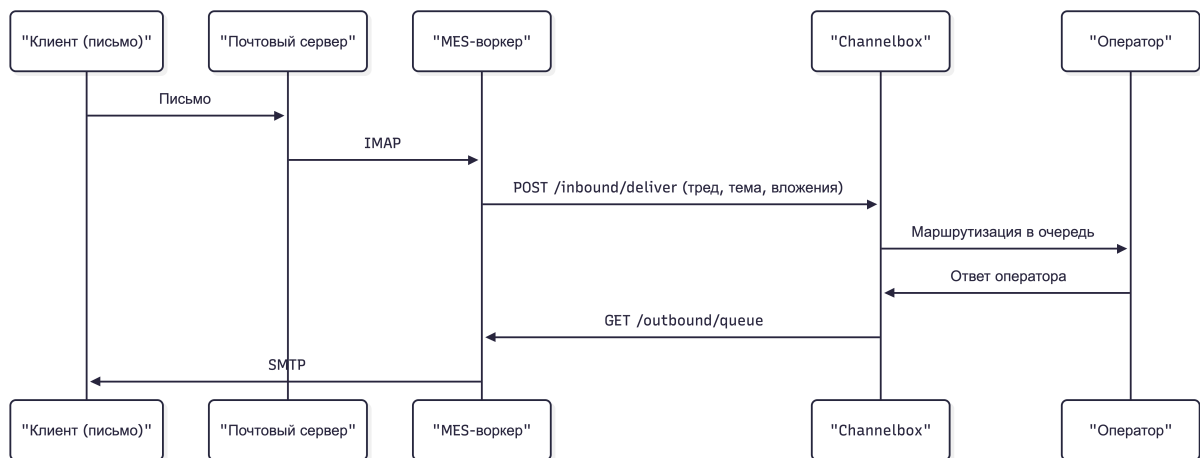
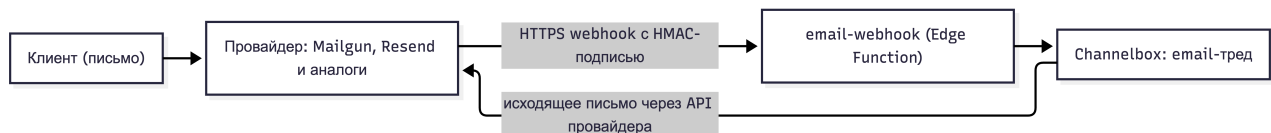


Преимущества токенизации:

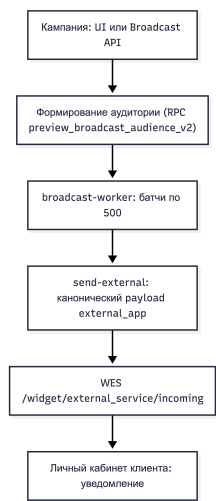
- PII не передаётся через WebSocket-канал
- Клиент хранит только UUID-токен
- TTL ограничивает время жизни данных в памяти
- Соответствие 152-ФЗ и GDPR

3.3 Email-обращение через MES

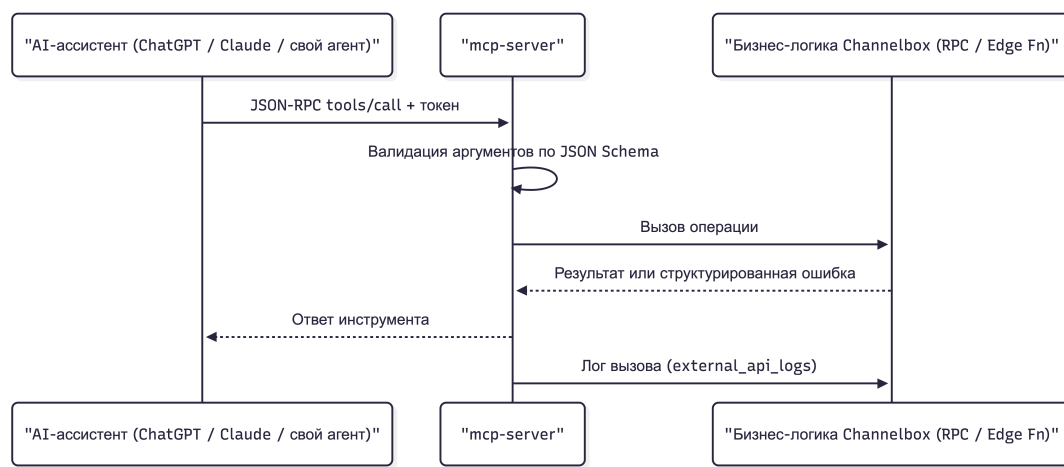
Основной путь — собственный воркер MES (SMTP/IMAP в контуре заказчика). Альтернатива — email webhook внешнего сервиса (Mailgun, Resend и аналоги), когда почта уже обслуживается провайдером.

**Альтернативный путь через провайдера:**

3.4 Массовая рассылка в личный кабинет (WES)



3.5 Вызов инструмента внешним AI-агентом (MCP)



4. Конфигурация виджета

Channelbox управляет конфигурацией внешних виджетов через Edge Functions:

Операция	Edge Function	Описание
Создать канал	<code>widget-external-service-api</code>	Регистрация канала с <code>secret_key</code> , <code>callback_url</code> , <code>domain</code>
Обновить канал	<code>widget-external-service-api</code>	Изменение параметров существующего канала
Конфиг виджета	<code>widget-external-service-api</code>	JSON-конфигурация внешнего вида и поведения
Embed-скрипт	<code>widget-external-service-api</code>	Генерация JS-скрипта для встраивания на сайт

CORS для `/dist/` (обязательно для Studio-превью).

`Loader (/dist/loader/loader.js)` – это ES-module. Cross-origin `<script type="module">` без CORS-заголовков не выполняется в браузере, даже если файл отдан `200 OK`. Клиентский сайт работает потому, что страница и loader находятся на одном origin; Studio-превью живёт на чужом домене (`chatapp.*`), поэтому WES-хост должен явно разрешить cross-origin чтение. Настройка nginx и диагностика – в `docs/troubleshooting/wes-loader-cors.md`.

Параметры embed-скрипта:

Параметр	Описание
Project ID	UUID проекта, к которому подключается виджет
Язык	Язык интерфейса виджета: русский или английский
Открыт по умолчанию	Определяет, будет ли виджет развёрнут при загрузке страницы

Действия с сообщениями	Включение возможности редактирования, удаления и ответа на сообщения
Режим работы	Режим разработки (dev) или продуктивный (prod)
Режим авторизации	Ручная авторизация через форму виджета или автоматическая из внешней среды

5. Сетевые требования

5.1 Домены и разделение endpoints

Домен (пример)	Тип	Назначение	Доступ
app.example.com	Frontend	Панель оператора (UI)	Только через VPN
api.example.com	API	Kong API Gateway – обработка запросов от WES, внешних каналов, Edge Functions	Внутренняя сеть + Firewall
wes.example.com	WebSocket	WES – прокси для виджетов	DMZ (Интернет)
token.example.com	API	Token Service – токенизация PII (HTTPS)	DMZ (доступен виджету и WES)
mail.example.com	API	Mail External Service – воркер SMTP/IMAP	Внутренняя сеть заказчика

Важно: Frontend и API – это **разные endpoints**. Операторы работают исключительно с UI (app.*) через VPN и не имеют прямого доступа к API (api.*), базе данных или внутренним сервисам.

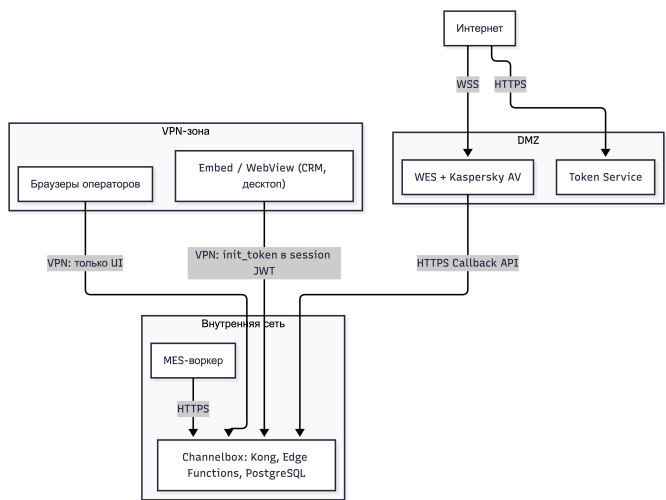
5.2 Порты

Порт	Сервис	Протокол
443	Nginx (Channelbox)	HTTPS
443	WES	WSS / HTTPS
443	Token Service	HTTPS

Все межкомпонентные вызовы проходят через HTTPS. PostgreSQL и внутренние сервисы Channelbox привязаны к `127.0.0.1` и не доступны извне.

5.5 Зоны доступа и VPN

Инфраструктура разделена на три изолированные зоны:



Зона	Компоненты	Доступ	Описание
VPN-зона	Браузеры операторов, Embed/WebView (CRM, десктоп-приложения)	VPN-туннель → <code>app.example.com</code>	Операторы и встроенные приложения подключаются к Frontend (UI) через корпоративный VPN. Доступ ограничен только интерфейсом — API, БД и внутренние сервисы недоступны
DMZ	WES + Kaspersky AV, Token Service	Интернет → WSS (<code>wes.example.com</code>) и HTTPS (<code>token.example.com</code>)	WES принимает WebSocket-подключения от виджетов, сканирует файлы, проксирует трафик во внутреннюю сеть. Token Service принимает HTTPS-запросы виджета на создание токена и отдаёт PII только WES
Внутренняя сеть	Channelbox (Kong, PostgreSQL, Edge Functions), MES-воркер	Только из DMZ и VPN-зоны	Недоступна из Интернета. Все сервисы привязаны к <code>127.0.0.1</code> , общение между компонентами — по HTTPS

Принцип: конечные пользователи (операторы) работают только с UI через VPN-канал. База данных, API и другие сервисы не выходят наружу, обеспечивая конфиденциальность данных.

5.6 Интеграция с внешними каналами и сервисами (Firewall)

Внешние каналы, сервисы и AI-провайдеры обмениваются данными с Channelbox через фиксированные endpoints, защищённые firewall.

Входящие webhook-и (внешний сервис → Channelbox)

Канал / Сервис	Edge Function	Endpoint	Верификация
Telegram	telegram-webhook	POST api.example.com/functions/v1/telegram-webhook	Проверка X-Telegram-Bot-API-Secret-Token
Viber	viber-webhook	POST api.example.com/functions/v1/viber-webhook	HMAC-SHA256 (X-Viber-Content-Signature)
Instagram	instagram-webhook	POST api.example.com/functions/v1/instagram-webhook	Проверка подписи запроса
Facebook	instagram-webhook (общий Meta-обработчик)	POST api.example.com/functions/v1/instagram-webhook	Проверка подписи запроса Meta
Email (провайдер)	email-webhook	POST api.example.com/functions/v1/email-webhook	HMAC-SHA256 (X-Webhook-Signature)
Email (свой воркер)	mail-external-service-api	POST api.example.com/functions/v1/mail-external-service-api/inbound/deliver	worker_secret хоста

Внешний бот	bot-gateway	POST api.example.com/ functions/v1/bot- gateway	X-Bot-Token / Authorization: Token ...
External Router (callback)	routing-assign	POST api.example.com/ functions/v1/routing- assign	X-API-Key с правами router

Исходящие запросы (Channelbox → внешние сервисы)

Направление	Edge Function / Компонент	Целевые хосты	Протокол
Мессенджеры	send-external	api.telegram.org , chatapi.viber.com , graph.facebook.com	HTTPS
Email	Email Edge Function	SMTP-сервер / Email API (Resend, MailForge)	SMTP / HTTPS
Outbound Webhooks (CRM)	outbound-webhook- dispatch	Настраиваемые URL (клиентские CRM, Service Desk)	HTTPS
System Webhooks	chat-events	Единый настраиваемый URL (system_settings)	HTTPS
External Router	chat-router	Настраиваемый URL внешнего роутера	HTTPS
AI-провайдеры	bot-process- message / bot-test	openrouter.ai/api , custom API endpoints	HTTPS
Внешние боты	bot-gateway	Настраиваемые URL ботов (endpoint_url)	HTTPS

Правила Firewall

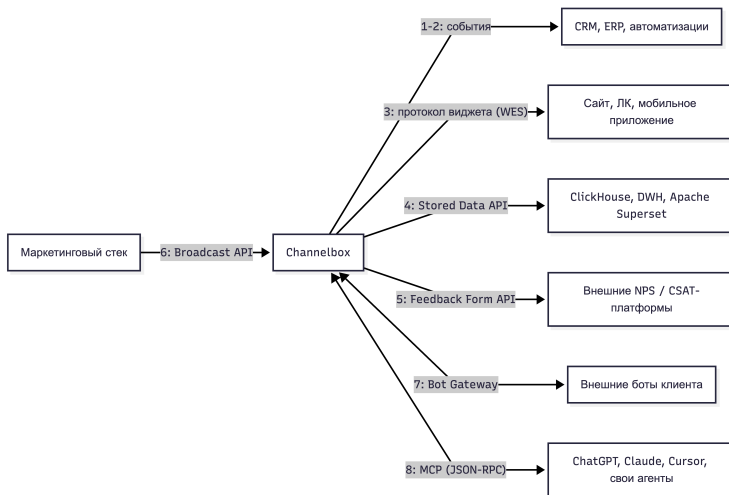
- **Входящие:** разрешены только POST-запросы на фиксированные endpoints (`/functions/v1/{channel}-webhook` , `/functions/v1/bot-gateway` , `/functions/v1/routing-assign`) с верифицированных IP-адресов / подписей
 - **Исходящие:** разрешены HTTPS-запросы от Channelbox к:
 - API мессенджеров (`api.telegram.org` , `chatapi.viber.com` , `graph.facebook.com`)
 - Почтовым провайдерам (SMTP, Resend API, MailForge API)
 - AI-провайдерам (`openrouter.ai` , custom LLM endpoints)
 - CRM / Service Desk (настраиваемые URL для Outbound/System Webhooks)
 - Внешним ботам и роутерам (настраиваемые URL)
 - **SSRF-защита:** Edge Functions блокируют запросы к приватным IP-диапазонам (`127.x` , `10.x` , `172.16.x` , `192.168.x`), `localhost` и внутренним доменам (`.internal` , `.local`)
 - Все остальные входящие и исходящие соединения блокируются
-

7. Интеграции

Channelbox предоставляет восемь независимых интеграционных поверхностей. Клиент подключает только нужные — поэтапно, без изменения остального ландшафта.

№	Поверхность	Направление	Раздел
1	Исходящие webhooks (бизнес-события)	Channelbox → внешние	7.2
2	Системные webhooks (синхронные, 5 с)	Channelbox → внешние	7.2
3	Протокол виджета (WES)	двусторонний	2.2, 4
4	Stored Data API (BI / DWH)	Channelbox → внешние	7.7
5	Feedback Form API (внешние формы обратной связи)	двусторонний	7.10
6	Broadcast API (массовые рассылки)	внешние → Channelbox	7.8
7	Bot Gateway (внешние боты)	двусторонний	7.6
8	MCP-протокол (инструменты для AI)	двусторонний	2.5, 7.9

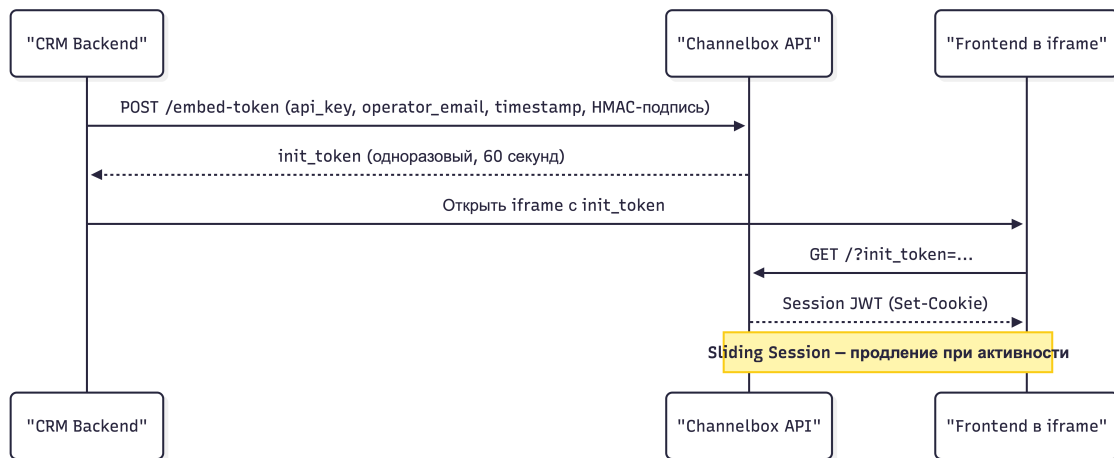
Дополнительно: Embed/WebView и CRM Screen-Pop (7.1, 7.12), Operator Status API (7.4), Operator Admin API (7.11), Consent API (7.13), BI-слой Apache Superset (7.14).



7.1 Embed / Iframe (MDbit Workspace)

Панель оператора может быть встроена в сторонние CRM и десктоп-приложения через `<iframe>` или Chromium-based оболочки (WebView / Electron).

Архитектура авторизации:



Ключевые характеристики:

- `init_token` – одноразовый, действует 60 секунд, обменивается на session JWT
- Sliding Session без third-party cookies – совместимо с Chromium-based оболочками
- Поиск оператора по email – case-sensitive (точное совпадение)
- Поддержка ограничения по `allowed_operator_ids` и `allowed_origins`

Поддерживаемые варианты встраивания:

Вариант	Описание
Iframe в CRM	Панель оператора как виджет внутри стороннего приложения
Chromium WebView	Полноценный десктоп-клиент на базе Chromium-движка
Electron App	Автономное приложение с embedded Channelbox UI

7.2 Outbound Webhooks и System Webhooks

Channelbox отправляет события во внешние системы через два механизма:

Outbound Webhooks (бизнес-события)

Настраиваются в **Настройки** → **Интеграции** → **Webhooks**. Поддерживают множество endpoint-ов с индивидуальными API-ключами.

Событие	Описание	Данные
<code>chat.started</code>	Новый диалог начат	Обогащённые данные разговора, посетителя, оператора, тегов
<code>chat.closed</code>	Диалог закрыт	Полный снимок: сообщения, теги, кастомные поля, причина закрытия
<code>visitor.updated</code>	Данные посетителя изменены	Снимок посетителя + текущий оператор с кастомными полями

Характеристики:

- Параллельная отправка через `Promise.allSettled` на все активные endpoint-ы
- API-версионирование (`v1`) для обратной совместимости
- Настраиваемый таймаут (по умолчанию 10 сек)
- Логирование результатов доставки в `outbound_webhook_logs`

System Webhooks (инфраструктурные события)

Настраиваются через `system_settings` (ключ `routing`). Единый URL, синхронная доставка.

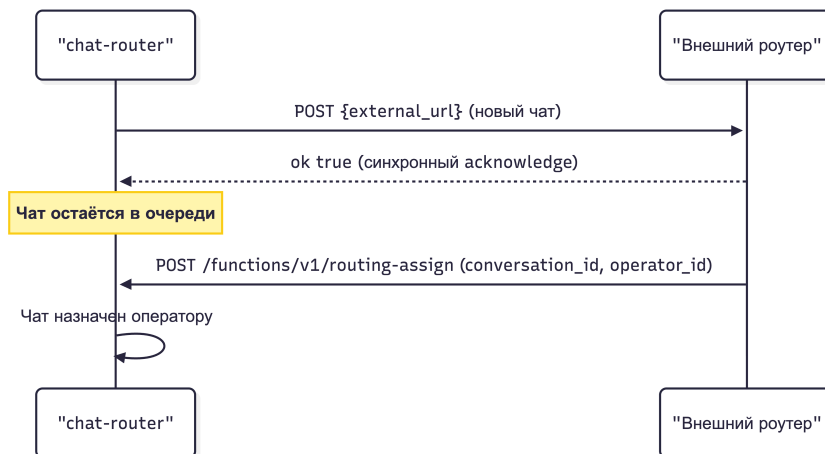
Событие	Описание
<code>operator.status_changed</code>	Оператор изменил статус (ready → break → offline)
<code>chat.assigned</code>	Чат назначен на оператора
<code>chat.queued</code>	Чат возвращён в очередь

Ограничения: таймаут 5 секунд, без повторных попыток. Для нестабильных внешних сервисов можно отключить через флаг `event_webhook_enabled`.

7.3 Внешний роутер (External Router)

Режим `distribution_mode = 'external'` позволяет делегировать маршрутизацию чатов внешнему сервису.

Асинхронный acknowledge-callback паттерн:



Защита от race conditions: перед применением состояния `QUEUE` проверяется текущий статус разговора — если callback уже назначил оператора, повторная запись в очередь не выполняется.

7.4 Operator Status API

Внешние системы могут управлять статусом оператора через Edge Function `routing-operator-status`.

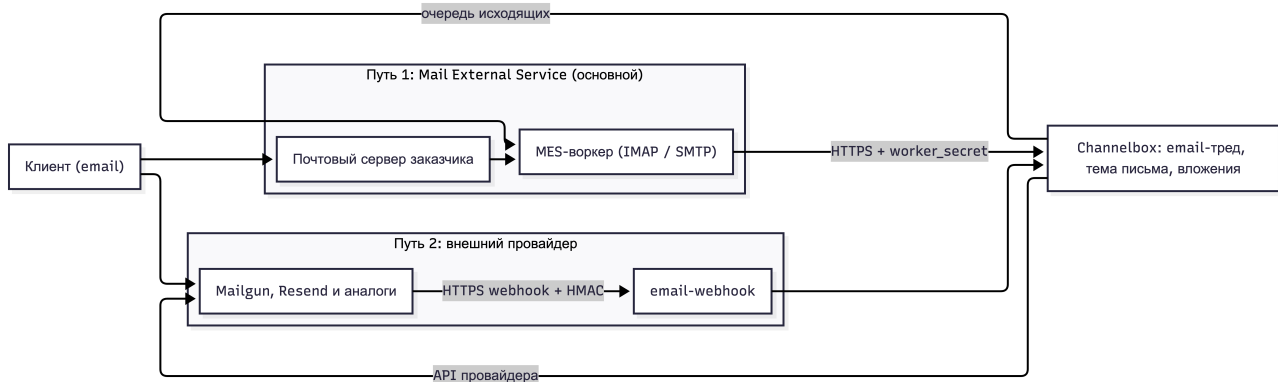
Параметр	Описание
Авторизация	X-API-Key с правами <code>router</code>
Идентификация	Только по <code>operator_id</code> (UUID)
Допустимые статусы	<code>ready</code> , <code>break</code> (требуется <code>break_reason_id</code>), <code>offline</code>

Автоматические действия при смене статуса:

- `break` / `offline` → активные чаты оператора автоматически возвращаются в очередь
- Изменение статуса инициирует System Webhook `operator.status_changed`

7.5 Email-канал

Channelbox поддерживает полноценный email-канал для приёма и отправки сообщений двумя путями: собственный воркер **Mail External Service** (основной для self-hosted) и **email webhook** внешнего провайдера (Mailgun, Resend и аналоги).



Входящие письма

Провайдер (Mailgun, Resend, MailForge) доставляет входящие письма через webhook:

Email провайдер → POST /functions/v1/email-webhook → Channelbox
(HMAC-SHA256)

- Автоматическое создание/обновление email-тредов
- Поддержка вложений (параллельная обработка)
- Case-insensitive нормализация заголовков

Исходящие письма

Используется Strategy-паттерн (`EmailProvider`) для поддержки нескольких провайдеров:

Провайдер	Тип	Описание
Resend	API	REST API для отправки email
Mailgun	API	REST API внешнего почтового сервиса
MailForge	API	Корпоративный email-сервис
SMTP	Протокол	Прямая отправка через SMTP-сервер
Mail External Service	Свой воркер	SMTP/IMAP в контуре заказчика, без облачных провайдеров (раздел 2.4)

Возможности:

- Email-threading (связывание ответов в цепочку)
- Конфигурируемые подписи с плейсхолдером `{name}`
- Опция `use_channel_sender_name` – скрытие PII оператора в заголовках

7.6 Боты и AI-провайдеры

Внешние боты (Callback API)

Channelbox поддерживает интеграцию с внешними ботами через `bot-gateway` :

Версия	Описание
v1.0	Базовый Callback API — текст, файлы, клавиатуры
v2.0	Расширенный — инлайн-действия, метаданные, <code>dialog_number</code>

Авторизация: `X-Bot-Token` для входящих запросов, проверка `secret_key` для исходящих.

Fallback при недоступности:

- 5 попыток с экспоненциальным backoff
- При окончательной недоступности: отправка `fallback_message` посетителю + перевод диалога в очередь

AI-провайдеры (LLM)

Channelbox использует provider-agnostic архитектуру для взаимодействия с внешними AI-сервисами:

Система определяет нужного AI-провайдера автоматически по идентификатору модели, указанному в настройках бота. Поддерживаются два типа провайдеров: OpenRouter (облачный агрегатор моделей) и пользовательские API-эндпоинты, совместимые с форматом OpenAI. Это позволяет использовать как облачные, так и локальные (on-premise) модели без изменения кода.

Характеристики:

- Динамическое разрешение провайдера по `model_id` через таблицы `ai_models` / `ai_providers`
- Единый OpenAI-совместимый маршрут через `handleOpenRouterBot`
- Шифрование API-ключей (`api_key_encrypted` , Base64)
- Поддержка custom base URL для on-premise LLM

7.7 Stored Data API — выгрузка данных для BI и CRM

Модуль `stored-data-api` предоставляет REST API для инкрементальной выгрузки данных из Channelbox во внешние аналитические системы. Спроектирован по образцу Webim Stored Data API v4, адаптирован под архитектуру Channelbox.

Назначение

Модуль предназначен для интеграции с системами бизнес-аналитики (ClickHouse, Power BI, Metabase), ETL-платформами (Airbyte), CRM-системами (Bitrix, AmoCRM) и Data Lake (S3, NDJSON/Parquet).

Авторизация

Используется существующий механизм API-ключей с заголовком X-API-Key. Для доступа к Stored Data API ключ должен иметь разрешение `stored_data` . Ограничение: 60 запросов в минуту на ключ.

Эндпоинты

Справочники (полная выгрузка)

Эндпоинт	Описание
----------	----------

GET /v1/operators	Список операторов: идентификатор, имя, email, статус, дата создания
GET /v1/queues	Очереди (отделы): идентификатор, название, порядок сортировки
GET /v1/categories	Категории диалогов с вложенностью
GET /v1/tags	Каталоги и теги

Диалоги (инкрементальная выгрузка)

Эндпоинт	Описание
GET /v1/chats?since={timestamp}	Диалоги, изменённые после указанного времени. Возвращает до 100 записей, курсор <code>last_ts</code> и флаг <code>more_available</code>
GET /v1/chat?id={uuid}	Один диалог по идентификатору

Каждый объект диалога содержит: метаданные (статус, очередь, канал, даты), массив сообщений, историю переходов статусов, привязанные теги, оценки и данные посетителя.

Посетители (инкрементальная выгрузка)

Эндпоинт	Описание
GET /v1/visitors?since={timestamp}	Посетители, изменённые после указанного времени

Статистика (агрегаты за день)

Эндпоинт	Описание
----------	----------

GET /v1/stats?date=YYYY-MM-DD&mode=usage	Общая статистика за день: количество обращений, закрытых, активных
GET /v1/stats?date=YYYY-MM-DD&mode=operators	Статистика по операторам: количество обработанных диалогов

Служебные

Эндпоинт	Описание
GET /v1/health	Статус API, версия, текущее время сервера

Паттерн инкрементальной синхронизации (CDC)

Для эндпоинтов `/chats` и `/visitors` используется курсорная пагинация через параметр `since` :

1. Первый вызов: `GET /v1/chats?since=0` – полная выгрузка
2. Из ответа извлекается значение `last_ts`
3. Следующий вызов: `GET /v1/chats?since={last_ts}` – только изменённые записи
4. Если `more_available = true`, повторять с новым `last_ts`
5. Потребитель выполняет `upsert` на своей стороне

Параметр `limit` позволяет настроить размер батча (по умолчанию 100, максимум 500).

Анонимизация PII

Опциональный параметр `?anonymize=true` маскирует персональные данные в ответе: email (первая буква + `***@домен`), телефон (звёздочки кроме последних 4 цифр), имя (сокращение фамилии). Используется для аналитических выгрузок, не требующих идентификации конкретных посетителей.

Безопасность

Модуль использует только запросы на чтение (SELECT) и не может изменить данные в системе. Все вызовы логируются в таблицу `external_api_logs`. Для защиты от перегрузки применяется ограничение частоты запросов (rate limiting).

7.8 Broadcast API – массовые рассылки

Кампании создаются как в интерфейсе, так и из внешнего маркетингового стека.

Возможность	Описание
Аудитория	Сегменты, импортированные списки, «конкретные посетители»; предпросмотр охвата через RPC
Каналы доставки	Мессенджеры, email, личный кабинет клиента (WES-режим уведомлений)
Канонический payload	Единая структура <code>external_app: type, priority, content</code> , вложенные элементы тела сообщения
Режим открытия ссылок (LOM-1)	Атрибут <code>open_mode</code> у каждой ссылки: внутренний раздел приложения или внешний ресурс
Надёжность	Батчи по 500 получателей, авто-пауза кампании при доле ошибок доставки выше порога, passthrough-режим для прямых получателей

7.9 MCP-протокол

Внешние AI-ассистенты подключаются к платформе как к MCP-серверу и получают 23 инструмента (см. 2.5). Обратное направление также поддерживается: платформа может вызывать MCP-серверы заказчика для обогащения ответов бота.

Авторизация: отдельный токен MCP с ротацией; каждый инструмент выполняется в контексте прав, заданных для интеграции.

7.10 Feedback Form API – внешние формы обратной связи

Контракт для сторонних систем, собирающих обратную связь (NPS/CSAT, формы на сайте заказчика).

Аспект	Решение
Авторизация	API-ключ + HMAC-подпись запроса по формуле из контракта

Вложения	До 10 файлов по 10 МБ через временные предподписанные ссылки
Жизненный цикл	Обработка бота, эскалация на оператора, закрытие, переоткрытие обращения

Полный контракт: docs/integrations/FEEDBACK-FORM-API.md.

7.11 Operator Admin API – управление учётными записями

Server-to-server CRUD по операторам для кадровых систем и IdM.

Возможность	Описание
Операции	Создание, изменение, блокировка операторов, смена пароля, назначение очередей и ролей
Авторизация	API-ключ с отдельным разрешением; ограничение частоты запросов
Аудит	Все изменения фиксируются в журнале аудита

7.12 CRM Screen-Pop – автооткрытие карточки клиента

Позволяет CRM открывать карточку клиента в момент, когда оператор получает чат.

Механизм	Описание
Webhook	События <code>chat.assigned</code> , <code>chat.closed</code> , <code>chat.hold</code> уходят в CRM с контекстом диалога и оператора
<code>postMessage (integration:event)</code>	Для встроенного режима – событие передаётся из панели оператора в родительское окно / WebView
Триггеры	<code>auto_assign</code> , <code>operator_accept</code> , <code>takeover</code> / <code>transfer</code> , <code>tab_switch</code> , закрытие, удержание

Защита от нагрузки	Кэш настроек 30 секунд, circuit breaker на исходящие вызовы
--------------------	---

Руководство по встраиванию: docs/integrations/EMBED-WORKSPACE-GUIDE.md; ТЗ для разработчика заказчика: docs/integrations/EMBED-INTEGRATION-SOW.md.

7.13 Consent API – согласия 152-ФЗ / GDPR

Единая точка для внешнего центра согласий: тексты согласий передаются в платформу, привязка выполняется по идентификатору шаблона согласия, запросы верифицируются исходящим секретом.

7.14 BI-слой: Apache Superset

Отчётность может выноситься во внешний Superset поверх выгруженных или подключённых напрямую данных.

Элемент	Описание
Источник данных	Представления <code>vw_superset_*</code> (например, время в статусах операторов, АНТ по чатам)
Способ подключения	Прямое чтение представлений либо загрузка через Stored Data API в ClickHouse / DWH
Документация для BI-разработчика	Готовые SQL-запросы и описания виртуальных датасетов: docs/reports/superset/README.md, operator-aht.md
Требование к времени	Все выгрузки и отчёты приводятся к системному часовому поясу платформы

8. API-методы платформы (сводная матрица)

Поверхность	Точка входа	Авторизация	Основные методы
Каналы (входящие)	<code>/functions/v1/{telegram,viber,instagram,email}-webhook</code>	Секрет канала / HMAC-подпись	POST события канала
WES	<code>/functions/v1/widget-external-service-api</code>	<code>secret_key</code> канала	<code>list_channels</code> , <code>list_channels_by_project</code> , <code>settings_get</code> , <code>script</code> , <code>send</code>
WES (в сторону виджета)	<code>{host}/widget/callback/{channel_id}</code> , <code>{host}/widget/external_service/incoming</code>	<code>secret_key</code> / сервисный ключ	Сообщения, уведомления, рассылки
MES	<code>/functions/v1/mail-external-service-api</code>	<code>worker_secret</code> хоста	<code>/health/heartbeat</code> , <code>/config/domains</code> , <code>/inbound/mailboxes</code> , <code>/inbound/check</code> , <code>/inbound/deliver</code> , <code>/outbound/queue</code>
Маршрутизация	<code>/functions/v1/routing-assign</code>	X-API-Key (право <code>router</code>)	Назначение чата оператору (<code>callback</code> внешнего роутера)

Статусы операторов	<code>/functions/v1/routing-operator-status</code>	X-API-Key (право router)	Смена статуса по <code>operator_id</code>
Учётные записи	<code>/functions/v1/operators-admin-api</code>	X-API-Key (право администрирования)	CRUD операторов, смена пароля, очереди и роли
Внешние боты	<code>/functions/v1/bot-gateway</code>	X-Bot-Token	События диалога, ответы бота, инлайн-действия
MCP	<code>/functions/v1/mcp-server</code>	Токен MCP	<code>tools/list</code> , <code>tools/call</code> (23 инструмента)
Stored Data API	<code>/functions/v1/stored-data-api/v1/*</code>	X-API-Key (право <code>stored_data</code>)	<code>/operators</code> , <code>/queues</code> , <code>/categories</code> , <code>/tags</code> , <code>/chats</code> , <code>/chat</code> , <code>/visitors</code> , <code>/stats</code> , <code>/health</code>
Embed / Screen-Pop	<code>/functions/v1/embed-token + integration:event</code>	API-ключ + HMAC (<code>init_token</code> , 60 c)	Выдача <code>init_token</code> , события жизненного цикла чата
Исходящие webhooks	Настраиваемые URL заказчика	Индивидуальный ключ endpoint-a	<code>chat.started</code> , <code>chat.closed</code> , <code>visitor.updated</code>
Системные webhooks	Единый URL из <code>system_settings</code>	Секрет из настроек	<code>operator.status_changed</code> , <code>chat.assigned</code> , <code>chat.queued</code>
Согласия	Consent API	Исходящий секрет	Передача текстов и привязка согласий

9. Эксплуатация self-hosted: ограничения и обязательные настройки

Область	Требование
Realtime	Лимит порядка 100 событий/с на арендатора. Публикация <code>supabase_realtime</code> должна содержать только реально используемые таблицы; после изменения Replica Identity требуется перезапуск контейнера Realtime
Запрет деградации	Ошибки синхронизации Realtime устраняются в корне; опрос (polling) в интерфейсе как обходной путь не применяется
Хранилище	Обязательно задать <code>public_storage_base_url</code> , иначе медиа-ассеты виджета недоступны (Mixed Content / внутренний хост)
Edge Functions	На VPS <code>ANON_KEY</code> должен быть согласован с <code>JWT_SECRET</code> , иначе функции отвечают 401
Тяжёлые отчёты	Журнал регистрации и «Диалоги» используют выделенные RPC с проталкиванием фильтров и частичными индексами; при таймауте 57014 см. runbook по оптимизации
Регламентные задачи	Все периодические задачи выполняются нативным <code>pg_cron</code> и регистрируются в мониторинге cron
Часовой пояс	Интерфейс и выгрузки форматируются по <code>system_timezone</code> из настроек
Мониторинг	Порты Prometheus и Grafana слушают только <code>127.0.0.1</code> и публикуются наружу исключительно через reverse проxy с авторизацией
Восстановление	Работоспособность резервных копий подтверждается тестовым восстановлением на отдельный стенд не реже одного раза в месяц

10. Наблюдаемость, SIEM и резервное копирование

10.1 Наблюдаемость

Слой	Компонент	Источник данных	Примечание
Метрики приложения	Prometheus	Edge Function <code>security-metrics</code> (Prometheus-формат), health-эндпоинты сервисов	Рекомендуемое хранение – 30 дней
Метрики инфраструктуры	Prometheus	node-exporter, postgres-exporter, метрики Kong	На каждом сервере при multi-server-схеме
Логи	Vector → Loki	stdout всех Docker-контейнеров, структурные метрики Edge Functions	Альтернатива Loki – ELK
Визуализация и алерты	Grafana	Prometheus + Loki как data sources	Регистрация новых пользователей отключается

Порты мониторинга не публикуются во внешнюю сеть. Подробности и готовые дашборды – в руководстве по мониторингу.

10.2 SIEM

События безопасности выгружаются Edge Function `security-export` по pull-модели с курсорной пагинацией (интервал 5 минут). Целевая система – **Wazuh**; остальные поддерживаются как варианты.

Система	Способ приёма	Формат
Wazuh (целевая)	Внешняя команда (<code>command</code>) с интервалом 300 с, вызывающая скрипт экспорта	NDJSON или CEF

Splunk	HTTP Event Collector (HEC)	NDJSON / CEF
Elastic	<code>_bulk</code> API	NDJSON
QRadar	Log Source (Syslog)	Syslog RFC 5424 / CEF

Дополнительно доступна push-модель: `security-alerts` отправляет webhook при превышении настроенных порогов.

10.3 Резервное копирование

Механизм	Роль	Периодичность	Назначение
WAL-G	Основной	Полный бэкап — ежесуточно, архивация WAL — непрерывно	Физические бэкапы и восстановление на точку времени (PITR) в S3-совместимое хранилище
<code>pg_dump</code>	Дополнительный	По необходимости	Логический экспорт: перенос между стендами, выборочная выгрузка
Синхронизация файлов	Дополнительный	Ежесуточно	Вложения и медиа-ассеты хранилища

Целевые показатели восстановления:

Конфигурация	RPO (допустимая потеря данных)	RTO (время восстановления)
Pilot / Start	до 24 часов	1–2 часа
Standard	минуты (за счёт архивации WAL)	30–60 минут
Medium / Large	минуты	5–15 минут (репликация + PITR)

Ключи доступа к хранилищу бэкапов задаются переменными окружения и не хранятся в репозитории. Процедуры настройки, восстановления и регламент проверки — в руководстве по резервному копированию.

11. Sizing

Подробные конфигурации сайзинга описаны в документе «Описание архитектуры и Sizing». Краткая сводка дополнительных компонентов:

Компонент	vCPU	RAM	SSD	Примечание
WES	2	4 ГБ	20 ГБ	Масштабируется горизонтально
MES (почтовый воркер)	2	4 ГБ	20 ГБ	Зависит от объёма вложений
Token Service	1	2 ГБ	10 ГБ	In-Memory, ~1 КБ/токен

Ресурсы Channelbox – см. конфигурации Pilot / Start / Standard / Medium в основном документе по Sizing.

04. Database Schema

Схема базы данных PostgreSQL (Supabase). **120 таблиц + 11 views.**

Категории таблиц

Категория	Количество	Описание
Core	8	Основные сущности
Configuration	14	Настройки системы
Routing	3	Маршрутизация и авто-назначение
Logging	7	Логи и аудит
SLA	3	Service Level Agreement
Bots / MCP	9	Чат-боты, сессии, MCP-инфраструктура
Knowledge Base	6	База знаний
GDPR/Consent	3	Персональные данные
Security	10	Безопасность
Operators	13	Операторы, роли, капабилити, поля
AI Providers	2	AI-провайдеры и модели
Feedback	7	Оценки, опросы, feedback log
Embed	2	Iframe-встраивание
Custom Fields	3	Пользовательские поля

Email	1	Email-треды
Mail Worker	6	Собственный почтовый воркер
Borrower Profile	3	Профили заёмщиков (группы, layouts)
Widget / WES	4	Виджет и WES-конфигурация
Outbound Webhooks	2	Исходящие вебхуки
Dashboard/Workspace	3	Дашборд и интеграции
Templates	3	Файлы, действия, переводы
Broadcasts	6	Массовые рассылки
Other	6	Прочие таблицы
Views	11	Представления

Core Tables (8)

- `conversations` – основная таблица чатов (статусы: pending, active, bot, hold, post_processing, closed)
- `messages` – сообщения (поле `is_internal` для внутренних заметок)
- `operators` – операторы системы
- `queues` – очереди обработки чатов
- `visitors` – посетители (клиенты)
- `message_attachments` – вложения к сообщениям
- `message_reactions` – реакции на сообщения
- `message_edit_history` – история редактирования

Configuration Tables (14)

- `external_integrations` – внешние каналы (consent_template_id, delivery_mode, channel_type)

- `operator_queues` — связь операторов с очередями
- `operator_roles` — роли (admin, supervisor, operator)
- `chat_tags` — теги (иерархические, с каталогами)
- `tag_catalogs` — каталоги тегов
- `conversation_tags` — связь тегов с чатами
- `message_templates` — шаблоны сообщений
- `message_template_categories` — категории шаблонов
- `message_template_tags` — связь шаблонов с тегами
- `break_reasons` — причины перерыва
- `work_schedules` — рабочие расписания
- `holidays` — праздничные дни
- `system_settings` — глобальные настройки (key-value)

Routing Tables (3)

- `routing_rules` — правила внешней маршрутизации (per-integration)
- `internal_routing_rules` — правила внутренней маршрутизации между очередями/ботами/операторами
- `auto_assign_logs` — журнал авто-назначения чатов (события `sys.auto_assign`)

Logging Tables (7)

- `external_api_logs` — логи внешних API
- `external_channel_logs` — логи внешних каналов
- `event_webhook_logs` — логи webhook событий
- `chat_history` — история состояний чата
- `cron_job_logs` — логи cron-задач
- `router_test_logs` — логи тестирования роутера
- `anonymization_logs` — логи анонимизации (GDPR)

Operators Tables (13)

- `operator_status_history` — история статусов
- `operator_notification_settings` — настройки уведомлений
- `operator_read_cursors` — позиции прочтения
- `pinned_conversations` — закреплённые чаты

- `queue_supervisors` – супервайзеры очередей
- `queue_tag_catalogs` – связь очередей с каталогами тегов
- `custom_roles` – кастомные роли (сверх admin/supervisor/operator)
- `custom_role_capabilities` – связь ролей с капабилити
- `capability_catalog` – реестр капабилити (действия/разрешения)
- `operator_capabilities` – прямые капабилити оператора
- `operator_custom_roles` – назначение кастомных ролей оператору
- `operator_field_definitions` – определения EAV-полей оператора
- `operator_field_values` – значения EAV-полей оператора
- `operator_sidebar_settings` – настройки правой сайдбар-панели оператора

SLA Tables (3)

- `sla_policies` – политики SLA
- `sla_tracking` – отслеживание SLA
- `sla_events` – события SLA (breach, warning)

Bots / MCP Tables (9)

- `bots` – чат-боты (ai, external, script)
- `bot_conversations` – сессии ботов
- `bot_prompt_versions` – версии промптов
- `bot_knowledge_links` – связь ботов с базами знаний
- `bot_sessions` – state machine сессии
- `dialog_sessions` – сессии диалогов (browser, geo, device info)
- `mcp_api_tokens` – токены для MCP-доступа ботов к инструментам
- `mcp_call_log` – лог вызовов MCP-инструментов
- `mcp_rate_limits` – rate limits для MCP

Knowledge Base Tables (6)

- `knowledge_bases`, `knowledge_pages`, `knowledge_blocks`, `knowledge_chunks`, `knowledge_tags`, `knowledge_page_tags`

GDPR/Consent Tables (3)

- `consent_templates`, `subject_consent`s, `consent_audit_log`

Security Tables (10)

- `security_api_keys`, `security_ip_bans`, `security_blocked_requests`, `security_metrics_hourly`, `security_alert_thresholds`, `security_alert_webhooks`, `security_autoban_rules`, `allowed_widget_domains`, `widget_rate_limits`, `auth_audit_log`

auth_audit_log – централизованный журнал аудита безопасности:

- Колонки: `id`, `created_at`, `event_type`, `severity` (default 'info'), `user_id`, `session_id`, `email`, `source_ip`, `user_agent`, `metadata` (JSONB), `category` (default 'auth')
- INSERT только через Edge Function `auth-audit-log` с `service_role` (обходит RLS)
- Категории: `auth`, `operator`, `settings`, `dialogs`, `reports`

AI Providers Tables (2)

- `ai_providers` – провайдеры (type: openai/anthropic/custom, api_key_encrypted, base_url)
- `ai_models` – модели (model_id, capabilities, provider_id)

Feedback Tables (7)

- `feedback_templates` – шаблоны оценок (scale_type: stars/nps/csat/thumbs/emoji)
- `chat_ratings` – оценки чатов (score, comment, dialog_id)
- `survey_templates` – шаблоны опросов
- `survey_questions` – вопросы опросов
- `survey_responses` – ответы на опросы
- `visitor_feedback_log` – лог запросов обратной связи
- `visitor_feedback_responses` – ответы посетителей (rating_value, rating_comment, response_type)

Embed Tables (2)

- `embed_api_keys` – API ключи для embed (`api_key`, `secret_key`, `allowed_origins`, `allowed_operator_ids`)
- `embed_sessions` – сессии embed (`init_token`, `expires_at`, `is_revoked`)

Custom Fields Tables (3)

- `visitor_field_definitions` – определения полей (`scope`: `visitor/conversation`, `field_type`, `is_required`)
- `visitor_field_values` – значения полей для посетителей
- `conversation_field_values` – значения полей для чатов (`scope`=`conversation`)

Email Tables (1)

- `email_threads` – email-треды (`original_message_id`, `references_chain`)

Mail Worker Tables (6)

Собственный почтовый воркер (self-hosted SMTP/IMAP): приём и отправка писем через собственные хосты и SMTP-аккаунты.

- `mail_worker_domains` – обслуживаемые почтовые домены
- `mail_worker_hosts` – IMAP/SMTP хосты. Флаг `allow_private_host` разрешает внутренние IP для self-hosted (в Cloud остаётся строго).
- `mail_worker_mailboxes` – почтовые ящики (креды, статус синхронизации)
- `mail_worker_smtp_accounts` – исходящие SMTP-аккаунты
- `mail_worker_inbound_log` – журнал входящих писем
- `mail_worker_outbound_queue` – очередь исходящих писем

Borrower Profile Tables (3)

Модуль профилей заёмщиков (кастомная бизнес-логика).

- `borrower_profile_groups` – группы полей профиля
- `borrower_profile_group_members` – состав групп
- `borrower_profile_layouts` – layout'ы отображения профиля

Widget / WES Tables (4)

- `allowed_widget_domains` – разрешённые домены встраивания виджета (см. также Security)
- `widget_rate_limits` – rate limits виджета (см. также Security)
- `widget_hosts` – конфигурация хостов виджета
- `wes_config_presets` – пресеты конфигурации Widget External Service

Outbound Webhooks Tables (2)

- `outbound_webhook_endpoints` – настройки эндпоинтов (url, events, api_key, timeout_ms)
- `outbound_webhook_logs` – логи отправок

Dashboard/Workspace Tables (3)

- `dashboard_settings` – настройки дашборда (user_id, settings JSON)
- `workspace_integrations` – интеграции workspace (module_id, config, position)
- `workspace_layouts` – layout виджетов

Templates Tables (3)

- `message_template_files` – файлы шаблонов
- `message_template_actions` – действия шаблонов сообщений (кнопки/интенты)
- `message_translations` – сохранённые авто-переводы сообщений (см. memory: Auto Translate)

Other Tables (6)

- `chat_categories` – категории чатов
 - `channel_users` – пользователи внешних каналов
 - `sticker_packs` – наборы стикеров
 - `system_setup_status` – статус первоначальной настройки
 - `link_preview_cache` – кэш превью ссылок
 - `viber_processed_tokens` – дедупликация обработанных Viber токенов
-

Broadcasts Tables (6)

Таблицы модуля массовых рассылок. Доступ – только роли `admin` и `supervisor` (RLS).

- `broadcast_campaigns` – кампании рассылок
 - `status`: `draft` | `scheduled` | `running` | `paused` | `completed` | `cancelled` | `failed`
 - `channel_type`: `telegram` | `email` | `widget` | `webim` | `external_app`
 - `audience_source`: `filter` | `list` | `combined`
 - `message_content` (JSONB), `audience_filter` (JSONB)
 - Счётчики: `total_recipients`, `sent_count`, `delivered_count`, `failed_count`
 - `throttle_rps` (default 10) – контроль скорости отправки
 - `broadcast_recipients` – получатели кампании
 - `visitor_id` (TEXT, не UUID) – поддержка внешних идентификаторов
 - `status`: `pending` | `sending` | `sent` | `delivered` | `failed` | `skipped`
 - `list_row_id` – ссылка на строку списка аудитории (nullable)
 - Уникальный индекс: (`campaign_id`, `visitor_id`)
 - `broadcast_logs` – журнал событий кампании
 - `event` (TEXT), `details` (JSONB)
 - INSERT только через `service_role` (Edge Functions), без RLS на INSERT
 - `broadcast_lists` – списки аудитории для рассылок
 - `column_mapping` (JSONB), `columns` (TEXT[])
 - `row_count`, `status` (`active/archived`)
 - `broadcast_list_rows` – строки списков аудитории
 - `visitor_id` (TEXT, nullable), `email` (TEXT, nullable)
 - `data` (JSONB) – произвольные данные строки
 - `broadcast_templates` – шаблоны сообщений рассылок
 - Хранение переиспользуемых форматов сообщений по каналам
 - `channel_type`, `format` (`plain_text/external_app/telegram/email`), `content` (JSONB), `is_active`
-

Views (11)

View	Описание
<code>conversations_with_visitor</code>	Чаты с данными посетителя
<code>chat_lifecycle_summary</code>	Сводка жизненного цикла чата
<code>operator_chat_stats</code>	Статистика чатов операторов
<code>operator_status_report</code>	Отчёт по статусам
<code>chat_status_durations</code>	Длительности статусов
<code>dialogs_view</code>	View для архива диалогов
<code>message_response_times</code>	Время ответа на сообщения
<code>operator_chat_utilization_report</code>	Утилизация операторов
<code>service_level_report</code>	Отчёт по уровню сервиса
<code>chat_stage_durations</code>	Длительности этапов чата
<code>dialog_lifecycle_summary</code>	Сводка жизненного цикла диалога

ER-диаграмма (Core)

```

erDiagram
    visitors ||--o{ conversations : has
    queues ||--o{ conversations : contains
    operators ||--o{ conversations : handles
    conversations ||--o{ messages : contains
    messages ||--o{ message_attachments : has
  
```

```

messages ||--o{ message_reactions : has

operators }o--o{ queues : assigned_to
operators ||--o{ operator_roles : has
operators ||--o{ operator_status_history : logs

queues ||--o| bots : uses
queues ||--o| work_schedules : follows
queues ||--o{ sla_policies : has

conversations ||--o{ chat_history : logs
conversations ||--o{ conversation_tags : tagged
conversations ||--o{ conversation_field_values : has
chat_tags ||--o{ conversation_tags : used_in

ai_providers ||--o{ ai_models : has
bots }o--o| ai_providers : uses

embed_api_keys ||--o{ embed_sessions : creates

visitors ||--o{ visitor_field_values : has
visitor_field_definitions ||--o{ visitor_field_values : defines
visitor_field_definitions ||--o{ conversation_field_values : defines

outbound_webhook_endpoints ||--o{ outbound_webhook_logs : logs

feedback_templates ||--o{ visitor_feedback_log : uses
survey_templates ||--o{ survey_questions : has
survey_templates ||--o{ visitor_feedback_log : uses

broadcast_campaigns ||--o{ broadcast_recipients : has
broadcast_campaigns ||--o{ broadcast_logs : logs
broadcast_campaigns }o--o| broadcast_lists : uses
broadcast_lists ||--o{ broadcast_list_rows : contains

custom_roles ||--o{ custom_role_capabilities : includes
capability_catalog ||--o{ custom_role_capabilities : referenced
operators ||--o{ operator_capabilities : has
operators ||--o{ operator_custom_roles : has
custom_roles ||--o{ operator_custom_roles : assigned_to
operator_field_definitions ||--o{ operator_field_values : defines

mail_worker_domains ||--o{ mail_worker_hosts : has
mail_worker_hosts ||--o{ mail_worker_mailboxes : hosts
mail_worker_mailboxes ||--o{ mail_worker_inbound_log : receives
mail_worker_smtp_accounts ||--o{ mail_worker_outbound_queue : sends

borrower_profile_groups ||--o{ borrower_profile_group_members : contains

messages ||--o{ message_translations : translated
message_templates ||--o{ message_template_actions : has

```

```
bots ||--o{ mcp_api_tokens : owns
mcp_api_tokens ||--o{ mcp_call_log : logs
```